

Lab Experiments 1-17

Data Handling & Visualization: Python Code & Output

Experiment 1

Student Average Scores

PYTHON

Code

```
import numpy as np

student_scores = np.loadtxt("Students_data.csv", delimiter=";", skiprows=1, usecols=(1,
2, 3, 4 )) subjects = ["Math", "Science", "English", "History"]

average_scores = np.mean(student_scores, axis=0)

highest_subject = subjects[np.argmax(average_scores)]

print("Average scores:", np.round(average_scores, 2))
print("Subject with highest average score:", highest_subject)
```

Output

Average scores: [76.33 80.5 76.67 80.67]

Subject with highest average score: History

Experiment 2

Past Month Price Average

PYTHON

Code

```
import pandas as pd

df = pd.read_csv("Sales_data.csv") average_price =
df[df["Month_sales"] == "November 2023"]["Price"].mean()
```

```
print("Average price in November 2023 (Past Month): ", round(average_price, 2))
```

Output

Average price in November 2023 (Past Month): 221.78

Experiment 3

House Average Price

PYTHON

Code

```
import numpy as np

house_data = np.loadtxt("House_data.csv", delimiter=";", skiprows=1)

houses_more_than_4 = house_data[house_data[:, 1] > 4]
average_price = np.mean(houses_more_than_4[:, 5])

print("Average price of houses with more than 4 bedrooms: ", round(average_price, 2))
```

Output

Average price of houses with more than 4 bedrooms: 409598.18

Experiment 4

Quarterly Sales Analysis

PYTHON

Code

```
import numpy as np

sales_data = np.genfromtxt("Sales_data.csv", delimiter=";", skip_header=1, dtype=str)

months = sales_data[:, 1] sales
= sales_data[:, 4].astype(float)
```

```

Q1 =
0.0
Q4 =
0.0

for i in range(len(months)):
    if ("January" in months[i]) or ("February" in months[i]) or
("March" in months[i]):
        Q1 += sales[i]
    elif ("October" in months[i]) or
("November" in months[i]) or ("December" in months[i]):
        Q4 += sales[i]

total_sales_year = np.sum(sales)

percentage_increase = ((Q4 - Q1) / Q1) * 100

print("Total sales for the year:", round(total_sales_year, 2))

print("Percentage increase from Q1 to Q4:", round(percentage_increase, 2), "%")

```

Output

Total sales for the year: 16884.79

Percentage increase from Q1 to Q4: 54.37 %

Experiment 5

Fuel Efficiency Analysis

PYTHON

Code

```

import numpy as np

fuel_data = np.genfromtxt('Fuel_data.csv', delimiter=',', skip_header=1, dtype=str)

fuel_efficiency = fuel_data[:, 7].astype(float)
make = fuel_data[:, 8]

average_efficiency =
np.mean(fuel_efficiency)
mazda_eff =
fuel_efficiency[make == "mazda"]
audi_eff = fuel_efficiency[make ==
"audi"]
mazda_avg =
np.mean(mazda_eff)
audi_avg =
np.mean(audi_eff)

```

```
percentage_improvement = ((mazda_avg - audi_avg) / audi_avg) * 100

print("Average Fuel Efficiency of all cars:", round(average_efficiency, 2),
      "MPG") print("Average Fuel Efficiency of Mazda:", round(mazda_avg, 2),
      "MPG") print("Average Fuel Efficiency of Audi:", round(audi_avg, 2), "MPG")

print("Percentage Improvement from Mazda to Audi:", round(percentage_improvement,
  2), "%")
```

Output

Average Fuel Efficiency of all cars: 29.33 MPG

Average Fuel Efficiency of Mazda: 30.1 MPG

Average Fuel Efficiency of Audi: 29.03 MPG

Percentage Improvement from Mazda to Audi: 3.67 %

Experiment 6

Customer Purchase Analysis

PYTHON

Code

```
import pandas as pd

df = pd.read_csv("grocerystore.csv") total_sales =
df["Sales"].sum() print("Total sales for the grocery store: ",
round(total_sales, 2))

discount = 0.10 tax = 0.18 price_after_discount = total_sales -
(total_sales * discount) price_after_tax = price_after_discount
+ (price_after_discount * tax)

print("Final amount after applying discount and tax: ", round(price_after_tax, 2))
```

Output

Total sales for the grocery store: 13948.83

Final amount after applying discount and tax: 14813.66

Experiment 7

Customer Order Analysis

PYTHON

Code

```
import pandas as pd

df = pd.read_csv("order_data.csv") total_orders =
df.groupby("Full Name")["Order Count"].sum()
avg_order_per_product = df.groupby("Items")["Order
Total"].mean() earliest_order = df["Order"].min()
latest_order = df["Order"].max()

print("Total number of orders made by each customer: ",
total_orders) print("Average order quantity for each product: ",
avg_order_per_product) print("Earliest order date: ", earliest_order)
print("Latest order date: ", latest_order)
```

Output

Total number of orders made by each customer: Full Name

Aarav Sharma 22

Anika Rao 15

Diya Patel 28

Kabir Khan 19

Meera Iyer 23

Rohan Gupta 25

Sara Reddy 28

Vikram Nair 21

Name: Order Count, dtype: int64

Average order quantity for each product: Items

Headphones 600.880909

Keyboard 793.656250

Laptop 721.737143

Monitor 763.680909

Mouse 812.640000

Tablet 971.296250

Webcam 924.258889

Name: Order Total, dtype: float64

Earliest order date: 2023-01-05

Latest order date: 2023-12-31

Experiment 8

Five Most Sold Products

PYTHON Code

```
import pandas as pd

df = pd.read_csv("Sales_data.csv") top_5_products =
df.groupby("Product_Name")["Price"].sum().nlargest(5)

print("Five most sold products: ", top_5_products)
```

Output

Five most sold products: Product_Name

Widget C 2947.47

Widget H 2748.86

Widget G 2298.22

Widget D 2193.02

Widget F 2007.40

Name: Price, dtype: float64

Experiment 9

House Data Analysis

PYTHON Code

```
import pandas as pd

df = pd.read_csv("house.csv")

location_price_average = df.groupby("Location")["price"].mean()
more_than_4beds = df[df["beds"] > 4].count()["beds"]
largest_sq_feet = df["size"].max()

print("Average house price for each location: ", location_price_average)
print("Number of houses with more than 4 bedrooms: ", more_than_4beds)
print("Largest house size: ", largest_sq_feet)
```

Output

Average house price for each location: Location

Downtown 499218.448333

Hillside 494531.001667

Riverside 508303.572500

Suburbs 433748.390000

Uptown 511786.931429

Name: price, dtype: float64

Number of houses with more than 4 bedrooms: 7

Largest house size: 4128

Experiment 10

Monthly Sales Visualization

PYTHON Code

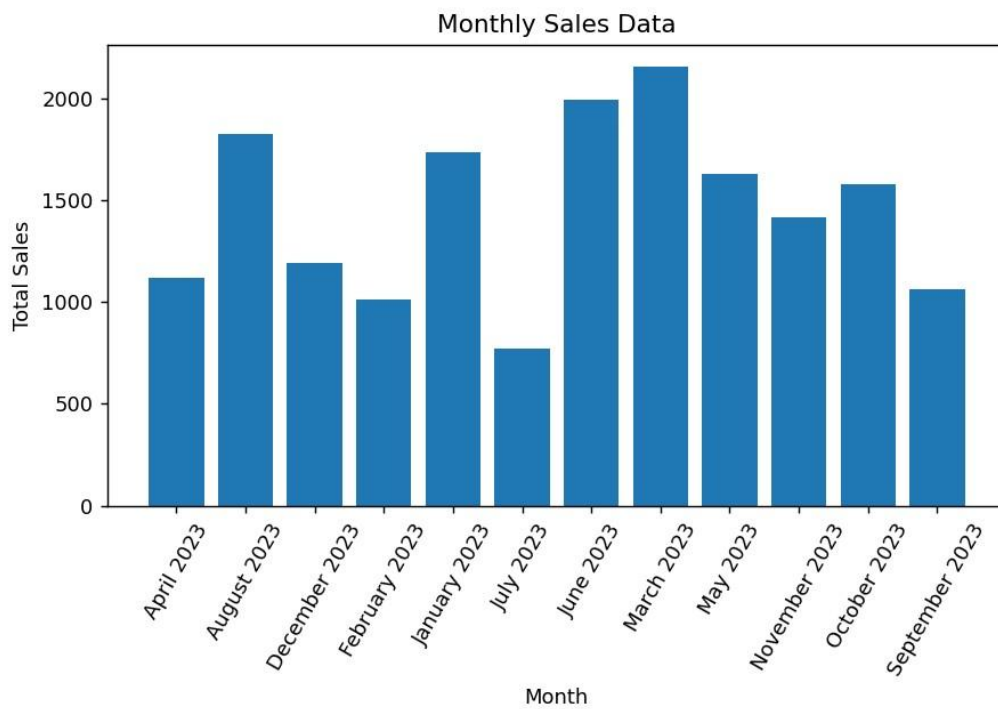
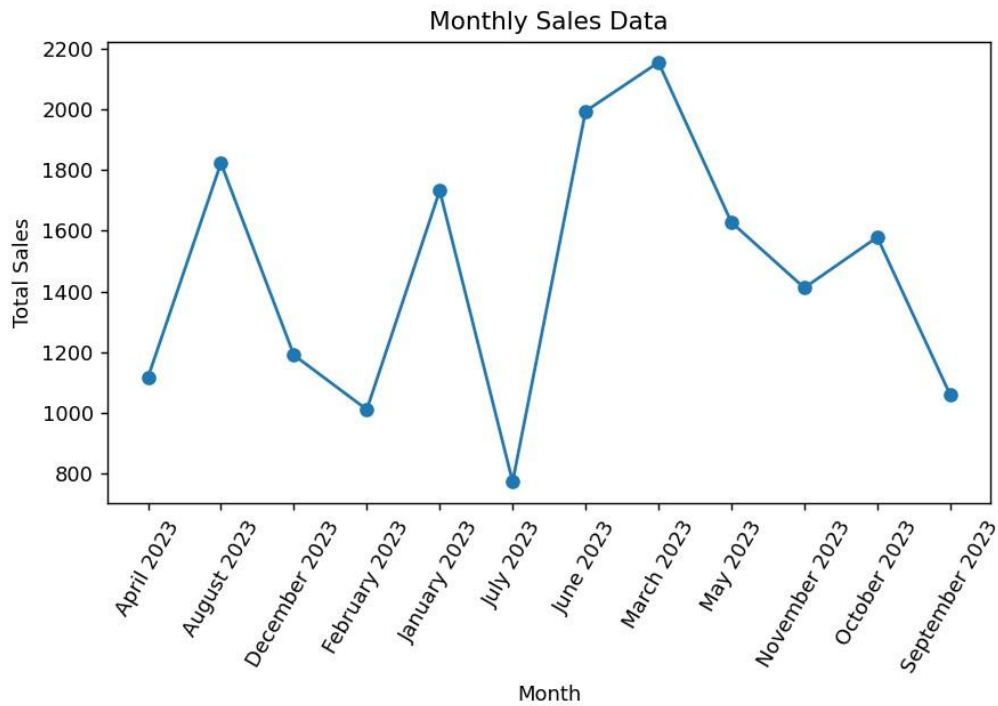
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('Sales_data.csv')
monthly_sales = df.groupby("Month_sales")["Price"].sum()
```

```
plt.plot(monthly_sales.index, monthly_sales.values, marker='o')  
plt.title("Monthly Sales Data") plt.xlabel("Month") plt.ylabel("Total Sales")  
plt.xticks(rotation=60) plt.show()
```

```
plt.bar(monthly_sales.index, monthly_sales.values)  
plt.title("Monthly Sales Data") plt.xlabel("Month") plt.ylabel("Total  
Sales") plt.xticks(rotation=60) plt.show()
```

Output



Lab Experiments 11-12

Data Handling & Visualization: Python Code & Output

Experiment 11

Monthly Sales Visualization

PYTHO

N

Code

```
import matplotlib.pyplot as plt

# Monthly sales data for a product (units sold) months =
['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct',
'Nov', 'Dec'] sales = [220, 245, 260, 210, 275, 300, 320, 290,
310, 330, 350, 400]

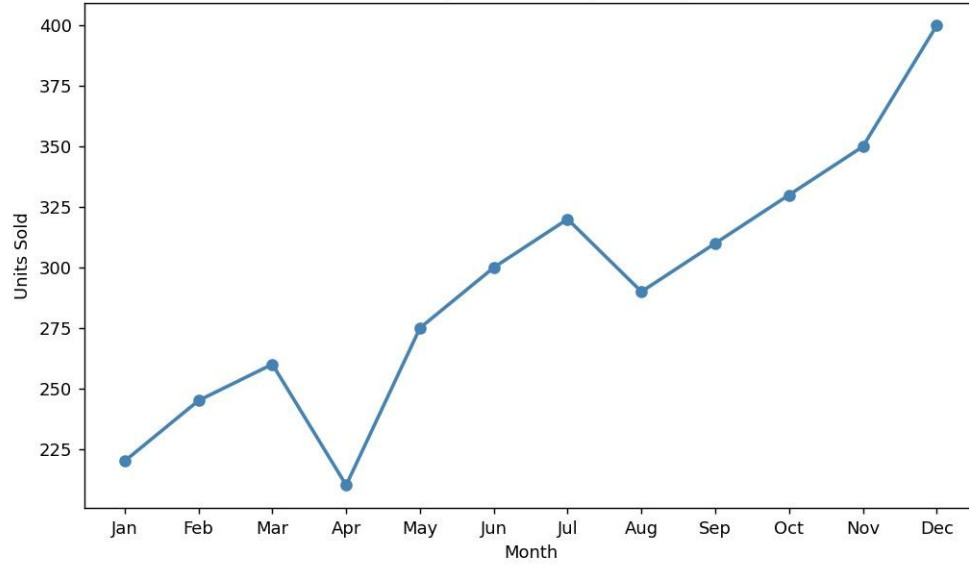
# 1. Line plot - monthly sales trend plt.figure(figsize=(8, 5))
plt.plot(months, sales, marker='o', color='steelblue',
linewidth=2) plt.title("Monthly Sales Trend (Line Plot)")
plt.xlabel("Month") plt.ylabel("Units Sold")
plt.tight_layout() plt.show()

# 2. Scatter plot - monthly sales
plt.figure(figsize=(8, 5))
plt.scatter(months, sales,
color='crimson', s=80) plt.title("Monthly
Sales (Scatter Plot)") plt.xlabel("Month")
plt.ylabel("Units Sold") plt.tight_layout()
plt.show()

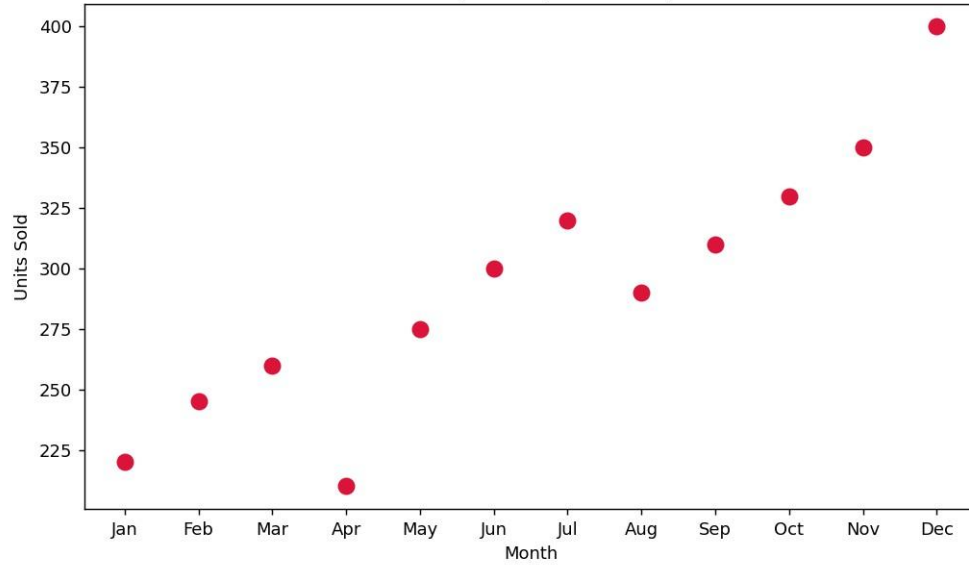
# 3. Bar plot - monthly sales data
plt.figure(figsize=(8, 5))
plt.bar(months, sales,
color='darkorange')
plt.title("Monthly Sales Data (Bar
Plot)") plt.xlabel("Month")
plt.ylabel("Units Sold")
plt.tight_layout() plt.show()
```

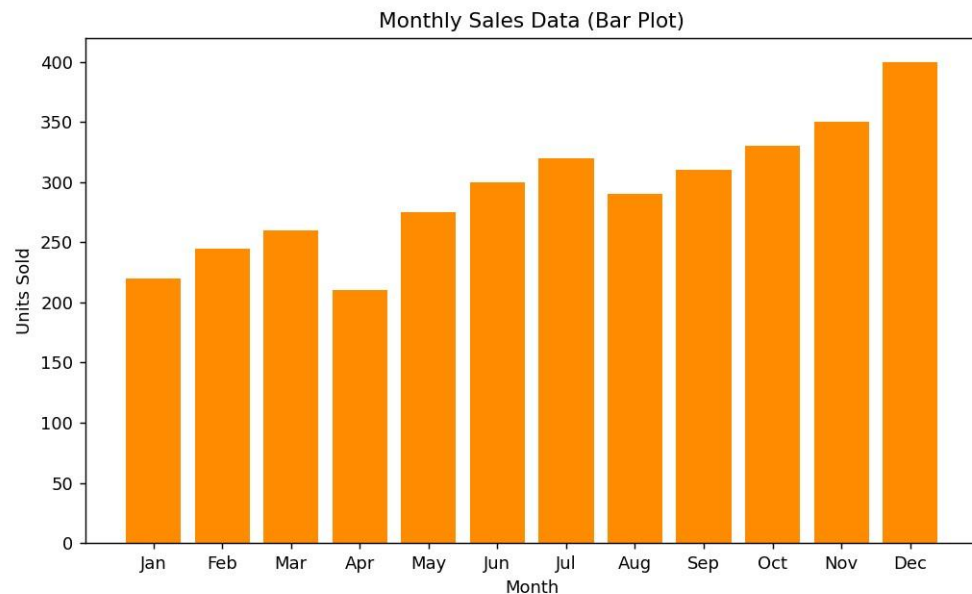
Output

Monthly Sales Trend (Line Plot)



Monthly Sales (Scatter Plot)





Experiment 12

Temperature & Rainfall Visualization

PYTHON Code

```
import matplotlib.pyplot as plt
```

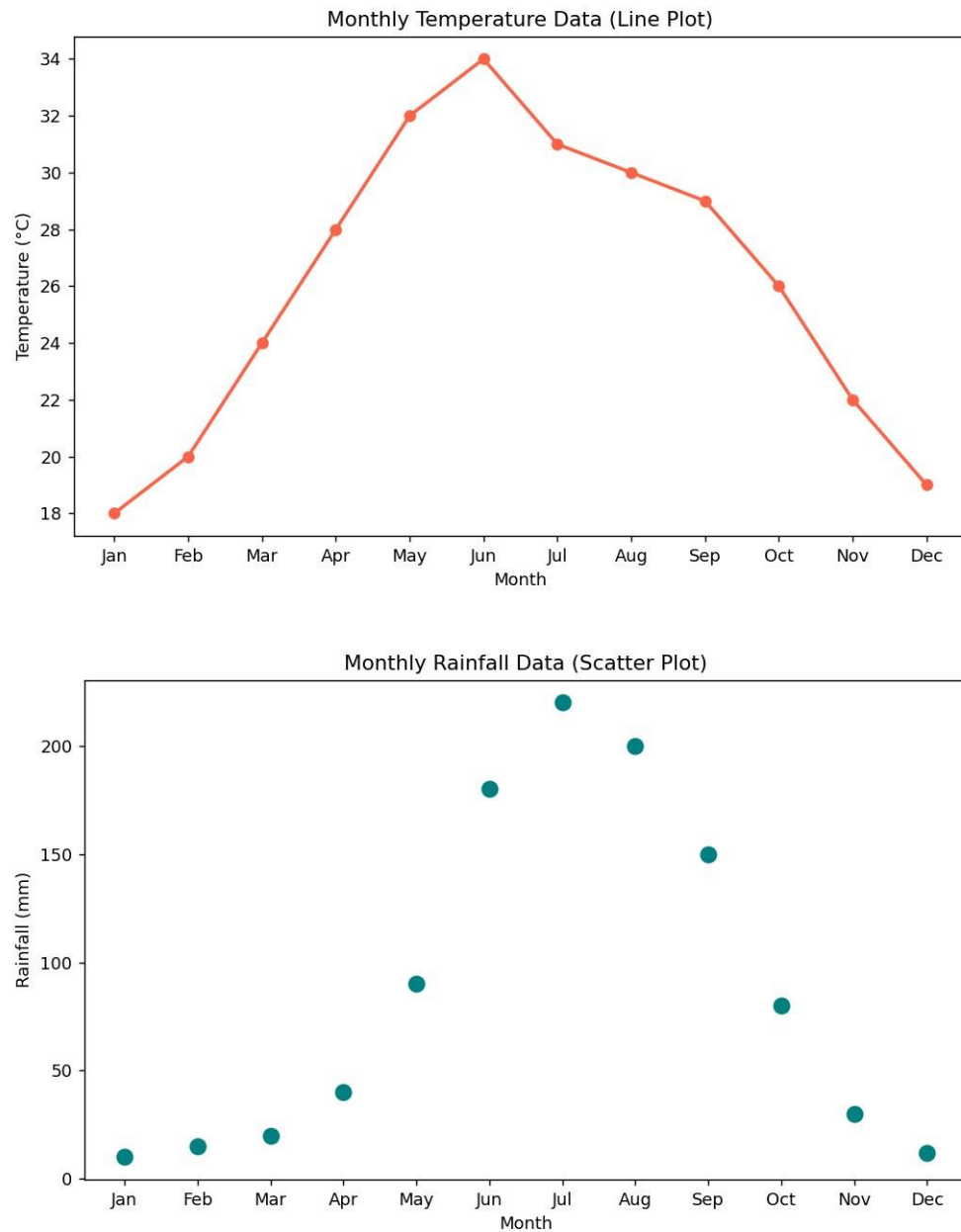
```
# Monthly average temperature data for a city (in Celsius) months = ['Jan', 'Feb',  
'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'] temperature = [18,  
20, 24, 28, 32, 34, 31, 30, 29, 26, 22, 19]
```

```
# Monthly rainfall data for the same city (in mm) rainfall = [10, 15, 20, 40, 90, 180,  
220, 200, 150, 80, 30, 12]
```

```
# 1. Line plot - monthly temperature data plt.figure(figsize=(8, 5)) plt.plot(months,  
temperature, marker='o', color='tomato', linewidth=2) plt.title("Monthly Temperature  
Data (Line Plot)") plt.xlabel("Month") plt.ylabel("Temperature (°C)") plt.tight_layout()  
plt.show()
```

```
# 2. Scatter plot - monthly rainfall data plt.figure(figsize=(8, 5))  
plt.scatter(months, rainfall, color='teal', s=80) plt.title("Monthly  
Rainfall Data (Scatter Plot)") plt.xlabel("Month") plt.ylabel("Rainfall  
(mm)") plt.tight_layout() plt.show()
```

Output



Experiment 13

Word Frequency Distribution from a Text Document

PYTHON Code

```
import string from collections import Counter
```

```
# Read the text document with  
open("sample_text.txt", "r") as file: text =  
file.read()
```

```
# Preprocess: lowercase and remove punctuation text = text.lower() text =  
text.translate(str.maketrans("", "", string.punctuation))  
  
# Split into words and count frequency words =  
text.split() word_freq = Counter(words)  
  
# Display the frequency distribution (most common first)  
print("Word Frequency Distribution:") for word, freq in  
word_freq.most_common(): print(f"{word}: {freq}")
```

Output

Word Frequency Distribution:

data: 12 and: 8 science: 5 to: 4
is: 3 good: 3 uses: 2 insights: 2
from: 2 the: 2 of: 2

an: 1 interdisciplinary: 1 field:
1 that: 1 scientific: 1 methods:
1 processes: 1 algorithms: 1
systems: 1 extract: 1
knowledge: 1 structured: 1
unstructured: 1 related: 1
mining: 1 machine: 1 learning:
1 big: 1 techniques: 1
theories: 1 drawn: 1 many: 1
fields: 1 within: 1 context: 1
mathematics: 1 statistics: 1
computer: 1 information: 1 a:
1 scientist: 1 collects: 1
cleans: 1 analyzes: 1 builds: 1
models: 1 help: 1
organizations: 1 make: 1
better: 1 decisions: 1
foundation: 1 every: 1
decision: 1 leads: 1

Experiment 14

Frequency Distribution of Customer Ages

PYTHON Code

```
import pandas as pd

# Sales data for the past month, stored in a Pandas DataFrame data = {
'CustomerID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12],

  'Age': [25, 34, 25, 41, 34, 28, 25, 34, 41, 30, 28, 25],

  'Product': ['A', 'B', 'A', 'C', 'B', 'A', 'C', 'B', 'A', 'C', 'B', 'A'],  'Amount': [500, 750, 500, 1200,
750, 450, 1200, 750, 500, 900, 450, 500]} df = pd.DataFrame(data)

# Frequency distribution of customer ages age_freq =
df['Age'].value_counts().sort_index()

print("Frequency Distribution of Customer Ages:") print(age_freq)
```

Output

Frequency Distribution of Customer Ages:

Age

25 4

28 2

30 1

34 3

41 2

Name: count, dtype: int64

Experiment 15

Frequency Distribution of Likes Among Posts

PYTHON Code

```
import pandas as pd

# User interaction data: number of likes received by each post data = {
'PostID': list(range(1, 13)),
```

```
'Likes': [120, 85, 120, 200, 85, 45, 120, 200, 85, 60, 45, 120] } df =  
pd.DataFrame(data)  
  
# Frequency distribution of likes among the posts likes_freq =  
df['Likes'].value_counts().sort_index()  
  
print("Frequency Distribution of Likes Among Posts:") print(likes_freq)
```

Output

Frequency Distribution of Likes Among Posts:

Likes

45 2

60 1

85 3

120 4

200 2

Name: count, dtype: int64

Experiment 16

Word Frequency Distribution in Customer Reviews

PYTHON

Code

```
import string from collections import Counter  
  
# Customer reviews dataset for a product reviews = [  
"The product quality is amazing and delivery was fast",  
    "Delivery was late but the product quality is great",  
    "Great value for money, I love this product",  
    "The product broke after a week, poor quality",  
    "Fast delivery and amazing customer service",  
    "I love the design but the quality could be better",
```



```

"Amazing product, will definitely buy again",
"Poor packaging led to a damaged product on delivery",
"Great customer service and fast delivery every time",
"The price is high for the quality offered"
]

# Preprocess: lowercase and remove punctuation
word_freq = Counter()
for review in reviews:
    review = review.lower().translate(str.maketrans("", "", string.punctuation))
    words = review.split()
    word_freq.update(words)

# Display the frequency distribution (most common first)
print("Word Frequency Distribution in Customer Reviews:")
for word, freq in word_freq.most_common():
    print(f"{word}: {freq}")

```

Output

Word Frequency Distribution in Customer Reviews:

```

the: 7
product:
6 quality:
5
delivery:
5 is: 3
amazing:
3 and: 3
fast: 3

great: 3

was: 2
but: 2 for:
2 i: 2
love: 2 a:
2 poor: 2
customer
: 2
service: 2

late: 1
value: 1

```

money: 1
this: 1
broke: 1
after: 1
week: 1
design: 1
could: 1
be: 1
better: 1
will: 1
definitely
: 1 buy: 1
again: 1
packagin
g: 1 led: 1
to: 1
damaged
: 1 on: 1
every: 1
time: 1
price: 1
high: 1
offered: 1

Experiment 17

Customer Feedback Word Frequency Analysis (Top N + Bar

Chart)

PYTHON

Code

```
import string
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
```

```
# Common stop words that don't carry significant meaning
```

```
STOP_WORDS = {
```

```
    "the", "a", "an", "and", "is", "it", "of", "to", "in", "on", "for",
```

```
    "was", "were", "with", "this", "that", "i", "my", "but", "at", "as",
```

```
    "are", "be", "so", "very", "has", "have", "had", "not", "will",  
    "would", "could", "can", "just", "if", "or", "we", "you", "your" }
```

```
def load_feedback(csv_path):  """Load the dataset from a CSV file  
containing a 'feedback' column."""  df = pd.read_csv(csv_path)  
return df["feedback"].astype(str).tolist()
```

```
def preprocess(text):  """Lowercase, strip punctuation, and remove  
stop words."""  text = text.lower().translate(str.maketrans("", "",  
string.punctuation))  words = text.split()  return [w for w in words if w  
not in STOP_WORDS]
```

```
def compute_word_frequency(feedback_list):  
    """Calculate the frequency distribution of words across all feedback  
entries."""  freq = Counter()  for entry in feedback_list:
```

```
    freq.update(preprocess(entry))  
return freq
```

```
def plot_top_n(freq, n):  """Plot a bar graph of the top  
N most frequent words."""  top_words =  
freq.most_common(n)  words, counts =  
zip(*top_words)
```

```
    plt.figure(figsize=(8, 5))  plt.bar(words, counts,  
color="darkorange")  plt.title(f"Top {n} Most Frequent Words  
in Customer Feedback")  plt.xlabel("Word")  
plt.ylabel("Frequency")  plt.xticks(rotation=45)  
plt.tight_layout()  plt.show()
```

```
if __name__ == "__main__":
```

```
    feedback_list = load_feedback("data.csv")  
    word_freq =  
    compute_word_frequency(feedback_list)
```

```
    n = int(input("Enter the number of top words to display: "))
```

```
    print(f"\nTop {n} Most Frequent Words:")  
for word, count in  
word_freq.most_common(n):  
    print(f"{word}: {count}")  
  
    plot_top_n(word_freq, n)
```

Output

Enter the number of top words to display:

5 Top 5 Most Frequent Words:

quality: 7

product: 6

delivery: 6

amazing: 4

fast: 4

